

BayesianFitForecast

From options file to posterior forecasts

Gerardo Chowell

A joint work with Hamed Karami, Amanda Bleichrodt, and Ruiyan Luo
Department of Population Health Sciences
Georgia State University

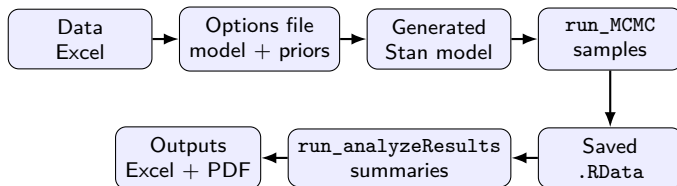
July 2026

Audience: graduate students and researchers who want to fit ODE models with Bayesian uncertainty quantification without writing Stan code manually.

By the end, users should be able to:

- prepare a time-series dataset and an options file;
- let `BayesianFitForecast` generate and run the Stan model;
- interpret posterior credible intervals, forecast prediction intervals, and diagnostics;
- compare Poisson, negative binomial, and normal observation models.

Workflow: from data to posterior forecasts



Main message

The user writes an options file; the toolbox generates Stan, runs Bayesian fitting, and exports diagnostics and forecasts.

Under the hood: `stancreator.R`

Purpose

`stancreator.R` reads the user-defined options file and writes the generated Stan model, `ode_model.stan`.

- Translates `vars` and `params` into Stan state variables and parameters.
- Converts `ode_system` into the Stan ODE function.
- Inserts priors into the Stan model block.
- Uses `fitting_index`, `fitting_diff`, and `errstrc` to build the likelihood.
- Uses `forecastinghorizon` to generate posterior predictions.

Main message

Users edit the options file; `stancreator.R` builds the Stan model automatically.

- ODE models are widely used to describe epidemic dynamics and produce forecasts in near real time.
- Bayesian calibration adds parameter uncertainty and forecast uncertainty to these model-based predictions.
- The main barrier is implementation: users often need to write Stan code, run MCMC, and organize output files manually.

Why this toolbox?

BayesianFitForecast turns a user-defined options file into a complete Bayesian fitting and forecasting workflow.

- Stan is a probabilistic programming language for Bayesian statistical modeling.
- In a standard workflow, users must define priors, likelihoods, parameters, and generated quantities directly in Stan.
- `BayesianFitForecast` generates the Stan model from the user-defined options file.
- It exports posterior summaries, convergence diagnostics, fit/forecast plots, parameter histograms, and performance metrics.

Two views of the parameter θ

- Frequentist analysis treats θ as fixed but unknown; the goal is to estimate it from data.
- Bayesian analysis updates prior information about θ using observed data.
- The prior density is $p(\theta)$ and the posterior density is $p(\theta \mid Y)$.
- The posterior distribution summarizes uncertainty after seeing the data.

Bayesian update

Bayesian inference estimates plausible parameter values given observed data and prior information.

$$p(\theta \mid Y) = \frac{p(Y \mid \theta)p(\theta)}{p(Y)}$$

Posterior density $\propto$ **likelihood** $\times$ **prior density**

- **MCMC** draws samples from the posterior distribution when direct sampling is difficult.
- **HMC** uses gradient information to explore posterior distributions efficiently.
- **Stan** uses HMC/NUTS to produce posterior samples, which are summarized by the toolbox.

Diagnostic reminder

Check R-hat, effective sample size, trace plots, divergent transitions, and posterior predictive checks.

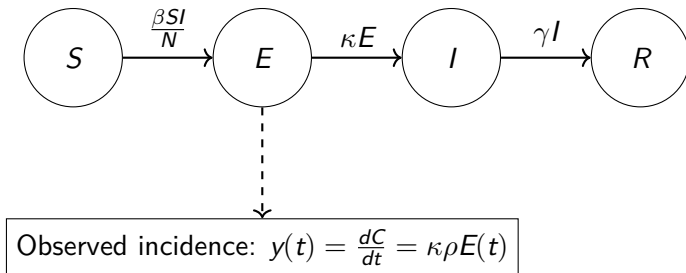


Figure: Compartmental diagram of the SEIR model with underreporting. Circles show the epidemiological compartments: susceptible (S), exposed (E), infectious (I), and recovered (R). Solid arrows indicate the transitions between compartments. The dashed arrow indicates the source of the observations, which are the newly reported cases.

SEIR Model

Consider the following model,

$$\frac{dS}{dt} = -\frac{\beta SI}{N}$$

$$\frac{dE}{dt} = \frac{\beta SI}{N} - \kappa E$$

$$\frac{dI}{dt} = \kappa E - \gamma I$$

$$\frac{dR}{dt} = \gamma I$$

$$\frac{dC}{dt} = \kappa \rho E$$

$C(t)$ is cumulative reported cases; dC/dt is incidence when `fitting_index` = 5 and `fitting_diff` = 1. Parameters: transmission β , incubation κ , recovery γ , reporting ρ .

Downloading the Toolbox

- Repository: <https://github.com/gchowell/BayesianFitForecast>
- Click **Code** and choose **Download ZIP**, or clone the repository.
- Keep the folder structure unchanged so the scripts can find options, inputs, and output folders.



GitHub QR code

Installation and Setup

- Install R and RStudio. The current CRAN package can be installed directly in R.
- Current CRAN release: `BayesianFitForecast` version 1.1.0; R requirement: `R >= 3.5.0`.
- Direct CRAN route: `install.packages("BayesianFitForecast")` followed by `library(BayesianFitForecast)`.
- The older compatibility note about R versions prior to 4.5 and Stan versions prior to 2.36 applies to the archived tutorial scripts, not necessarily to the current CRAN release.
- On Windows, install Rtools. Java is needed only for workflows that use Java-dependent Excel packages.
- Install required packages when using the tutorial workflow: `bayesplot`, `readxl`, `xlsx`, `openxlsx`, `rstan`, `ggplot2`, `loo`, `stringr`, `gridExtra`, and `dplyr`.

The prior_sltn.R Function

Purpose

Generates **prior predictive simulations** by sampling from user-defined prior distributions.

Workflow

- 1 Draw samples from prior distributions
- 2 Solve the ODE system using each sample
- 3 Construct an ensemble of trajectories to reflect **prior uncertainty**
- 4 To use this function, you must define the ODE model—including parameters and state variables, and specify the prior distributions for the parameters within the options file. The structure and setup of this file will be explained in detail later.

Example: Strong Prior Distribution Using Simulated Data

- In this example, we review the solution generated by sampled prior distribution using the SEIR model described earlier.
- We generate the simulated data using $\beta = 0.6$, $\gamma = 0.1$, $\kappa = 0.2$, $\rho = 0.9$, and $N = 10,000$.
- We use a strong normal prior distribution for the parameters with mean around the true values. Then, the solutions should be around the actual data.

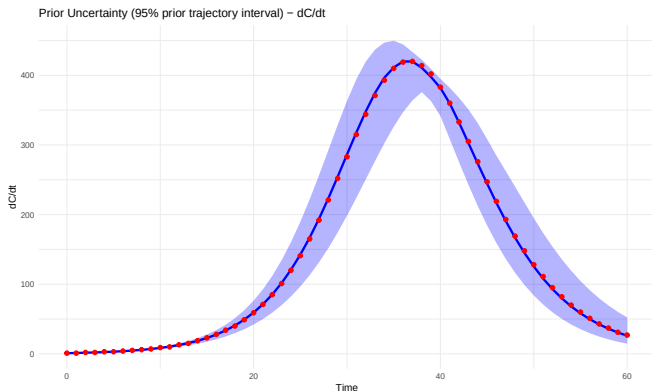
Prior Specification in R

```
params1_prior <- "normal(0.6, 0.01)T[0,]" # beta: transmission rate
params2_prior <- "normal(0.1, 0.01)T[0,]" # gamma: recovery rate
params3_prior <- "normal(0.2, 0.01)T[0,]" # kappa: incubation rate
params4_prior <- "normal(0.9, 0.01)T[0,]" # rho: reporting proportion
params5_prior <- 10000                      # N: fixed population size
Ic <- c(9990, 5, 5, 0, 0)
```

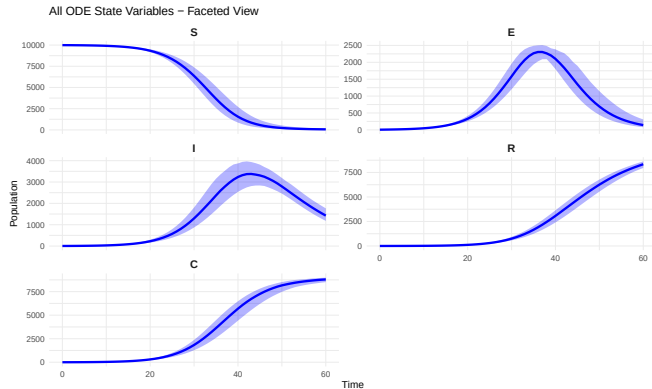
- Uses **informative priors** (low variance, $\sigma = 0.01$)

Prior Predictive Simulation: Incidence

- Tight prior distributions yield low variability in simulated incidence trajectories.
- These trajectories represent beliefs before observing the fitting data.
- Use Bayesian terminology: prior uncertainty is summarized by a 95% prior trajectory interval.



Prior Predictive Simulation: State Variables



Takeaway: prior choices imply a family of SEIR trajectories.

Main Objective

Estimate the **posterior distribution** of plausible parameter values given the observed data and prior assumptions.

What's Next

To achieve this, we must first fully explain the structure and role of the **options file**.

Next: Posterior Distribution

- We now want to **combine prior knowledge with observed data**
- Goal: obtain **posterior distributions** of parameters
- This requires the **options file** configuration

**Reviewing all settings in one
option file**

Definition: Calibration periods are cutoff points used to fit the model to the first part of the time series.

- Enter them as an **integer vector in R**.
- Each value is an end point/cutoff, not a full interval.
- Even one calibration period should be written as a vector.

Examples

```
calibrationperiods <- c(10, 15, 20, 25, 30) fits several cutoffs.
```

```
calibrationperiods <- c(10) fits only the first 10 data points.
```

Definition: `forecastinghorizon` is the number of future time points simulated after the calibration period.

- If `forecastinghorizon <- 10` during `run_MCMC`, post-processing can later report any horizon less than or equal to 10.
- The workflow consists of two scripts: `run_MCMC` is the slow sampling step; `run_analyzeResults` is the faster post-processing step.
- Use a larger horizon at sampling time if shorter forecast summaries may be needed later.

Adjusting the Forecasting Horizon

- To zoom in after sampling, change `forecastinghorizon` after `run_MCMC.R` and before `run_analyzeResults.R`.
- Positive values request forecasts. A value of 0 is used for fitting-only output.

Practical rule

Run MCMC with the largest horizon you need; summarize shorter horizons later.

Definition: `model_name` assigns a project label and helps avoid overwriting results.

- Use distinct names when priors, calibration periods, or likelihoods differ.
- Avoid spaces, slashes, backslashes, and special symbols in filenames.

Safe naming convention

`model-disease-error-cal-fcst-date`

Example: `seir-sf1918-poisson-cal17-fcst10-20250825`

State Variables in the SEIR Model

Variable	Meaning
S	Susceptible
E	Exposed
I	Infectious
R	Recovered
C	Cumulative reported cases

Order matters

`vars <- c("S", "E", "I", "R", "C")` must match the order of derivatives in `ode_system`.

Definition: The parameters in the SEIR model represent values that affect the dynamics of the system. They can be either constants or values to be estimated during model fitting.

Parameters:

- "beta": Transmission rate
- "gamma": Recovery rate
- "kappa": Incubation rate
- "rho": Reporting proportion
- "N": Total population size

Parameters Definition: All parameters must be included in the vector `params`, and each parameter should be defined as a string.

Example:

```
params <- c("beta", "gamma", "kappa", "rho", "N")
```

Time-Dependent Parameters

- Time-dependent parameters are defined in the `time_dependent_templates` object.
- Each is a string function of time `t` and elements from the `params` vector.

Example: Exponential Decay

```
time_dependent_param1 <- "  
return (params1 * exp(-params2 * t));  
"
```

Parameter mapping

Here `params1` is the initial value or scale, and `params2` is the exponential decay rate.

Piecewise Linear

```
time_dependent_param2 <- "  
if (t < params1) {  
  return(params2);  
} else if (t < params3) {  
  return(params2 + (params4 -  
    params2) *  
    (t - params1) / (params3 - params1  
      ));  
} else {  
  return(params4);  
}"
```

Mathematical Expression

$$\beta(t) = \begin{cases} \beta_0 & \text{if } t < t_1 \\ \beta_0 + (\beta_1 - \beta_0) \frac{t-t_1}{t_2-t_1} & \text{if } t_1 \leq t < t_2 \\ \beta_1 & \text{if } t \geq t_2 \end{cases}$$

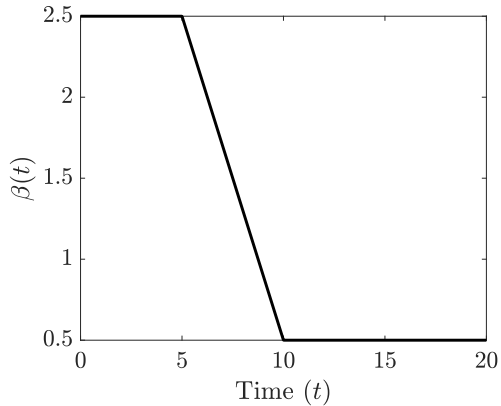


Figure: Piecewise linear function with the parameters $\beta_0 = 2.5$, $\beta_1 = 0.5$, $t_1 = 5$, and $t_2 = 10$.

Overview: `ode_system` defines the ODE right-hand sides using indexed parameter and variable names.

Code name	Meaning
<code>paramsi</code>	the i -th entry of <code>params</code>
<code>varsi</code>	the i -th entry of <code>vars</code>
<code>diff_vari</code>	derivative of the i -th state variable

Order check

If `vars` or `params` are reordered, every indexed term in the ODE code must be checked.

Example: Assume the following definitions for variables and parameters:

```
vars <- c("S", "E", "I", "R", "C")
```

```
params <- c("beta", "gamma", "kappa", "rho", "N")
```

Then, the ODE system can be defined as:

```
ode_system <- '
```

```
diff_var1 = -params1 * vars3 * vars1 / params5
```

```
diff_var2 = params1 * vars3 * vars1 / params5 - params3 * vars2
```

```
diff_var3 = params3 * vars2 - params2 * vars3
```

```
diff_var4 = params2 * vars3
```

```
diff_var5 = params4 * params3 * vars2'
```

Fixing Parameters in the Model

Use `paramsfix` to decide which parameters are fixed and which are estimated.

Parameter	<code>paramsfix</code> value	Status
β	1	fixed
γ	0	estimated
κ	0	estimated
ρ	0	estimated
N	1	fixed

Rule

1 = fixed; 0 = estimated. The vector length must match `params`.

Example for constant- β SEIR

```
composite_expressions <- list(  
  R0 = "beta/gamma",  
  recovery_time = "1/gamma"  
)
```

Assumption

$R_0 = \beta/\gamma$ assumes constant β . If $\beta(t)$ is time-dependent, state the assumption or use a time-varying metric.

fitting_index and fitting_diff

Definition

`fitting_index` selects the model variable matched to the data. `fitting_diff` determines whether the data are fitted to the variable itself or to its derivative.

Data target	<code>fitting_index</code>	<code>fitting_diff</code>	Fits to
Infectious prevalence	<code>c(3)</code>	<code>c(0)</code>	$I(t)$
Reported incidence	<code>c(5)</code>	<code>c(1)</code>	$\frac{dC}{dt}$
Two data streams	<code>c(3,5)</code>	<code>c(0,1)</code>	$I(t)$ and $\frac{dC}{dt}$

SEIR note

In the SEIR example, $C(t)$ is cumulative reported cases, so $\frac{dC}{dt} = \rho\kappa E(t)$ represents reported incidence.

Code	Error structure	When to use
1	Negative binomial	overdispersed count data
2	Normal	continuous data or Gaussian approximation
3	Poisson	count data where conditional variance equals conditional mean

Negative binomial parameterization

$$Y_t \mid \Theta, \phi \sim NB(\mu_t, \phi), \quad E(Y_t) = \mu_t, \quad \text{Var}(Y_t) = \mu_t + \mu_t^2 / \phi.$$

Larger ϕ means less overdispersion; smaller ϕ means more overdispersion.

cadfilename1: Excel filename without the .xlsx extension.

Column	Required name	Meaning
1	any time label	days, weeks, years, etc.
2	cases1	first observed time series
3	cases2	optional second time series
4+	cases3, ...	additional time series

caddisease: disease/process label used in the output folder name.

series_cases: This character vector defines the time-series label(s) for the y-axis.

```
series_cases <- c("infectious", "recovered")  
datatype <- "Days"
```

datatype: This parameter specifies the x-axis label. In this example, the x-axis is labeled Days.

Definition: This section allows users to specify prior distributions for parameters. If a parameter is fixed (indicated by "paramsfix" being 1), its prior is a constant value. For estimable parameters, enter a distribution as a string.

Parameter Considerations: Pay attention to the range of parameters. For instance, parameters like β , γ , and κ are positive. When using normal distributions, truncate at 0 (e.g., $\mathcal{T}[0,]$).

Priors - Example

Example: Given:

- `params <- c("beta", "gamma", "kappa", "rho", "N")`

Prior beliefs:

- 1 `beta`: 2 (constant)
- 2 `gamma`, `kappa`: `"normal(1,2)T[0,]"` (truncated normal)
- 3 `rho`: `"uniform(0,1)"` (uniform)
- 4 `N`: 10000 (constant)

The prior distribution setup:

- `params1_prior <- 2`
- `params2_prior <- "normal(1,2)T[0,]"`
- `params3_prior <- "normal(1,2)T[0,]"`
- `params4_prior <- "uniform(0,1)"`
- `params5_prior <- 10000`

Definition: This section allows users to define lower and upper bounds for parameters, restricting the range of values during estimation.

Bounds:

- "LB": Minimum value a parameter can take.
- "UB": Maximum value a parameter can take.
- Use NA for no restrictions.

Lower and Upper Bounds - Example

Example: Given:

- `params <- c("beta", "gamma", "kappa", "rho", "N")`

Parameter bounds:

- ① **beta**, **gamma**, **kappa**: Lower bounds 0, no upper bounds.
- ② **rho**: Lower bound 0, upper bound 1.
- ③ **N**: No lower or upper bounds.

Bound definitions:

- `params1_LB <- 0, params1_UB <- NA`
- `params2_LB <- 0, params2_UB <- NA`
- `params3_LB <- 0, params3_UB <- NA`
- `params4_LB <- 0, params4_UB <- 1`
- `params5_LB <- NA, params5_UB <- NA`

Priors for Error-Structure Parameters

- Negative binomial likelihood estimates a positive dispersion parameter ϕ .
- Normal likelihood estimates a positive standard deviation σ .

Positive priors

Use truncated or half-Cauchy priors for positive parameters, for example

```
normalerror1_prior <- "cauchy(0, 2.5)T[0,]"
```

Which error prior is used?

errstrc	Likelihood	Prior needed
1	negative binomial	<code>negbinerror1_prior</code> , <code>negbinerror2_prior</code> , ...
2	normal	<code>normalerror1_prior</code> , <code>normalerror2_prior</code> , ...
3	Poisson	no extra error parameter

Reminder

Set lower bounds so $\sigma > 0$ and $\phi > 0$.

Initial Conditions (Ic)

Ic gives the initial values of the state variables in the same order as vars.

Numeric example

If $S(0) = N - i_0$, $E(0) = 0$, $I(0) = i_0$, $R(0) = 0$, $C(0) = i_0$, and $i_0 = 1$, then

```
Ic <- c(params5_prior - 1, 0, 1, 0, 1)
```

Symbolic example when vars.init = 0

```
Ic <- c("N-i0", 0, "i0", 0, "i0")
```

Purpose: Sometimes, the **initial conditions** (e.g., initial number of infectious or exposed individuals) are unknown and need to be estimated. This can be done by setting **vars.init = 0**. If the values are known, set **vars.init = 1**.

When vars.init = 0:

- **params**: Add the new parameter (e.g., i_0) to params.
- **paramsfix**: Update to include the new parameter, with its entry set to 0 (for estimation).
- **priors**: Define the prior distribution for the new parameter.
- **Ic**: Specify initial conditions. Example for SEIR:

```
Ic <- c("N-i0", 0, "i0", 0, "i0")
```

When vars.init = 1: No changes are needed.

Definition:

- **niter**: This specifies the **number of iterations** for MCMC sampling.
- **num_chain**: This sets the **number of MCMC chains**.

Importance: Both parameters are crucial in Bayesian analysis using Stan to ensure **convergence** of the MCMC algorithm.

Example: To set the values for 2000 iterations and 4 chains:

```
niter = 2000,    num_chain = 4
```

These values can be adjusted based on model complexity and diagnostic results. R-hat below a chosen threshold, such as 1.1, is useful, but it is only one convergence diagnostic and does not by itself guarantee reliable sampling.

Process after configuring the options file

- ① Edit the options file and check the model/data mapping.
- ② Run `run_MCMC.R` to generate the Stan file and posterior samples.
- ③ Save the resulting `.RData` files.
- ④ Run `run_analyzeResults.R` to export tables, diagnostics, and plots.

Purpose: The file `run_MCMC` is designed to `fit the model` to the data using user-defined parameters.

Configuration:

- The user must update the options in the `options file` as needed.
- The user must also ensure that `line 5` in the `run_MCMC` file matches the name of the configured options file.

Importance: Updating the options and ensuring correct file references is crucial for running the MCMC fitting process smoothly.

Output: After running `run_MCMC`, the generated samples will be saved as an `.Rdata` file in the toolbox folder.

File Naming: For each period in `calibrationperiods`, a corresponding `.Rdata` file is generated. The filename is based on:

- `model_name`
- `calibrationperiod`
- `forecastinghorizon`
- `errstrc`
- `caddisease`

Recommendation: If two option files share identical settings, the second run will `overwrite` the first file. It is recommended to modify `model_name` to avoid this.

- The file `run_MCMC_Parallel.R` is designed to **accelerate MCMC computation** by running multiple chains in parallel.
- Within the R environment, the `rstan` package utilizes R's built-in `parallel` package to run chains concurrently.
- Users can set the number of cores via:

```
options(mc.cores = parallel::detectCores())
```

- This command automatically detects the maximum number of available cores and assigns one core per chain.
- Significantly reduces computation time.
- Reduces wall-clock time by running chains concurrently. It does not make the Markov chains themselves converge faster.

Purpose: By running `run_analyzeResults`, users can generate and analyze results based on the previously configured options.

Key Points: Ensure the `option file` remains the same as in `run_MCMC` (except for the forecasting horizon). Modify line 13 to specify the option file name.

File Naming: Generated files will follow the format:

```
model_name + caddisease + errstrc +  
calibrationperiod + forecastinghorizon
```

Convergence File: Includes the convergence diagnostic **R-hat**. Use R-hat below the chosen threshold together with effective sample size, trace plots, divergent transitions, and posterior predictive checks. Increasing **niter** or **num_chain** can help, but it does not guarantee acceptable diagnostics.

Example format:

`convergence-Bayesian-weak-sanfrancisco-normal-cal-10-fcst-10`

Forecast File: Includes columns for Date, Data, Median, Lower/Upper Bounds of 95% PIs. Example format:

`forecast-Bayesian-weak-sanfrancisco-normal-cal-10-fcst-10`

Parameters File: Provides posterior mean, median, and 95% credible intervals for parameters. For normal error structure: σ , For negative binomial: ϕ .

Example format:

`parameters-Bayesian-weak-sanfrancisco-normal-cal-10-fcst-10`

Performance Metrics File: Includes MAE, MSE, WIS, and Coverage for calibration/forecasting periods. Example format: `performance`

`metrics-Bayesian-weak-sanfrancisco-normal-cal-10-fcst-10`

In addition to Excel files, several PDF files will be generated:

- **Histograms:** Save histograms of parameters, including σ or ϕ . Example: "beta-histogram-Bayesian-weak-sanfrancisco-normal-cal-10.pdf".
- **Forecast:** Shows median fit and 95% PI margins. Example: "Forecast-Bayesian-weak-sanfrancisco-normal-cal-10-fcst-10".
- **Trace plot:** Visualizes parameter values over MCMC iterations. Example: "traceplot-Bayesian-weak-sanfrancisco-normal-cal-10-fcst-10".

Example 1: Poisson error structure

Learning goal

Fit the SEIR model to incidence data using a Poisson likelihood and fixed epidemiological parameters. This example uses fitting-only output.

- Dataset: San Francisco 1918 influenza.
- Model target: new reported cases, represented by dC/dt .
- Under a Poisson observation model, the conditional variance equals the conditional mean.

How to start

Create `options_SEIR_sanfrancisco_Ex1.R` from `options_SEIR_sanfrancisco_normalprior.R`.

Key parameters in the new file

```
calibrationperiods <- c(17)
forecastinghorizon <- 0
vars <- c("S", "E", "I", "R", "C")
ode_system <- '
  diff_var1 = -params1 * vars3 * vars1 / params5
  diff_var2 = params1 * vars3 * vars1 / params5 - params3 * vars2
  diff_var3 = params3 * vars2 - params2 * vars3
  diff_var4 = params2 * vars3
  diff_var5 = params4 * params3 * vars2'
paramsfix <- c(0,1,1,1,1)
```

Important Settings

Ensure the following settings are correct for fitting and error structure:

- `fitting_index = 5`
- `fitting_diff = 1`
- `errstrc = 3`

Execution

To run the MCMC algorithm:

- Modify line 5 in `run_MCMC` to use the new options file:
`"options_SEIR_sanfrancisco_Ex1.R"`.
- Execute the script in RStudio and analyze results using `run_analyzeResults.R`.

Fitting using Poisson error structure

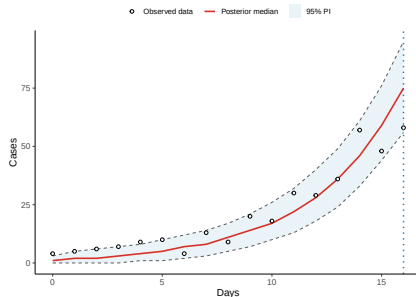
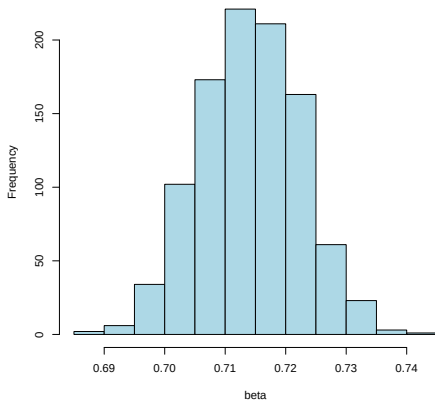


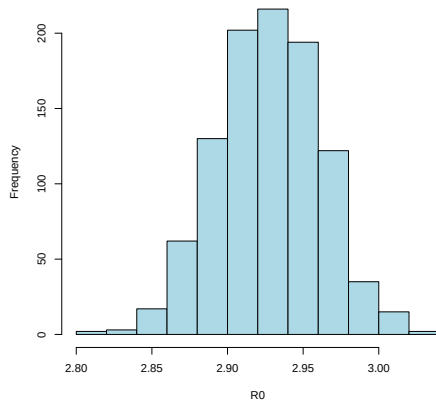
Figure: Poisson fit to the first 17 days of San Francisco 1918 flu incidence. Fixed parameters: $\kappa = 1/1.9$, $\gamma = 1/4.1$, $\rho = 1$, and $N = 550,000$; β is estimated with a uniform prior.

Histogram of posterior distributions: Poisson error structure

Median: 0.71, Mean: 0.71, 95% CrI: (0.70, 0.73)



R_0 – Median: 2.93, Mean: 2.93, 95% CrI: (2.86, 2.99)



Posterior inference and convergence analysis: Poisson error structure

Table: Posterior inference and convergence analysis for the example applying the SEIR model to the San Francisco 1918 influenza dataset with a Poisson error structure.

Calibration	Parameter	Mean	Median	95% CrI	Bulk ESS	R-hat
17	β	0.71	0.71	(0.70, 0.73)	316.35	1

Table: The performance metrics resulting from Bayesian fitting of the SEIR model to the 1918 influenza dataset from San Francisco, using a Poisson error structure and 17 days of calibration data.

	Calibration	Forecasting
MAE	5.24	NA
MSE	45.35	NA
WIS	3.33	NA
Coverage of 95% PI (%)	88.24	NA

Example 2: Poisson vs. negative binomial

Learning question

Does the negative binomial likelihood better account for overdispersion than the Poisson likelihood?

- Keep fixed parameters, priors, calibration period, and data identical.
- Only the observation error structure changes.
- Negative binomial intervals may be wider because ϕ allows extra-Poisson variability.

Fitting using negative binomial error structure

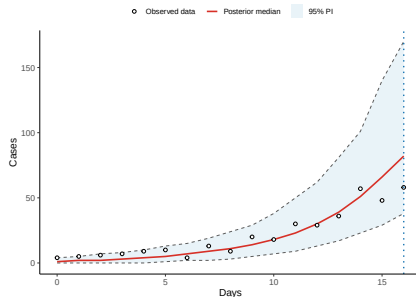
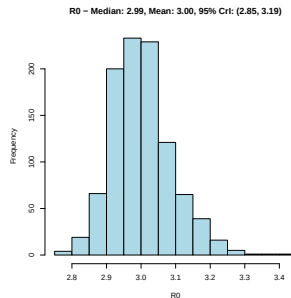
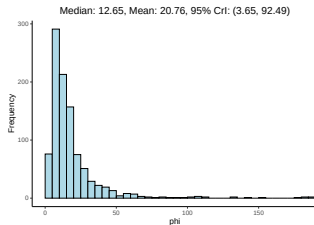
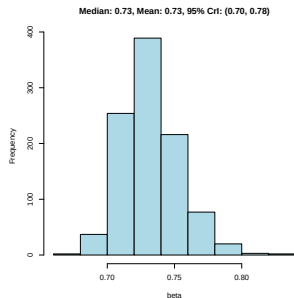


Figure: Negative binomial fit to the first 17 days of San Francisco 1918 flu incidence. Settings match the Poisson example except for the observation error structure and dispersion parameter ϕ .

Parameter histograms: negative binomial error structure

Under this negative-binomial parameterization, smaller ϕ corresponds to greater overdispersion.



Performance metrics: negative binomial error structure

Table: Performance metrics for the SEIR model with a negative binomial error structure, based on the San Francisco 1918 influenza dataset, calibrated over 17 days.

	Calibration	Forecasting
MAE	5.76	NA
MSE	68.24	NA
WIS	3.57	NA
Coverage of 95% PI (%)	100.00	NA

Convergence: negative binomial error structure

Table: Convergence and other statistics for the SEIR model applied to the San Francisco 1918 influenza dataset with a negative binomial error structure.

Parameter	Mean	Median	95% CrI	Bulk ESS	R-hat
β	0.73	0.73	(0.70, 0.78)	356.83	1
ϕ	20.76	12.65	(3.65, 92.49)	395.63	1

Table: Comparison of performance metrics between the Poisson and negative binomial error structures for the SEIR model, based on the SF 1918 influenza dataset.

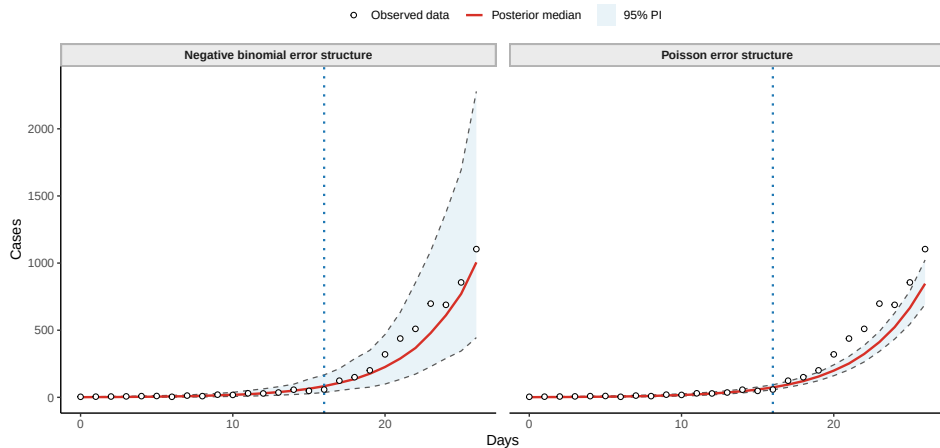
Model	MAE	MSE	Coverage of 95% PI (%)	WIS
Bayesian, Poisson error	5.24	45.35	88.24	3.33
Bayesian, Negative binomial error	5.76	68.24	100	3.57

Example 3: Forecasting with Poisson vs. negative binomial

In this example and under these metrics, the negative binomial model performs better in the forecast period after calibrating 17 days of the SF 1918 flu dataset. During the calibration period, the Poisson error structure performs better in terms of MAE, MSE, and WIS metrics, while the negative binomial model excels in 95% PI coverage. However, this conclusion is based on one calibration/forecast split and should be stated cautiously.

The forecast for both Poisson and negative binomial error structures is presented, along with a comparison of their performance metrics. The related option files for this example are provided as "options_SEIR_sanfrancisco_Ex3_Negbin" and "options_SEIR_sanfrancisco_Ex3_Poisson".

Forecast negative binomial vs. Poisson



Comparison of performance metrics in forecasting

Table: Comparison of Poisson and negative binomial performance. Calibration rows use the same fitting summaries reported in the previous examples; forecasting rows summarize the 10-day forecast run.

Error structure	MAE	MSE	Coverage of 95% PI (%)	WIS
Negative binomial, calibration	5.76	68.24	100.0	3.57
Poisson, calibration	5.24	45.35	88.24	3.33
Negative binomial, forecasting	92.10	12325.90	100.0	56.84
Poisson, forecasting	149.75	29976.22	10.0	122.28

Example 4: Comparison of SEIR and exponential growth models

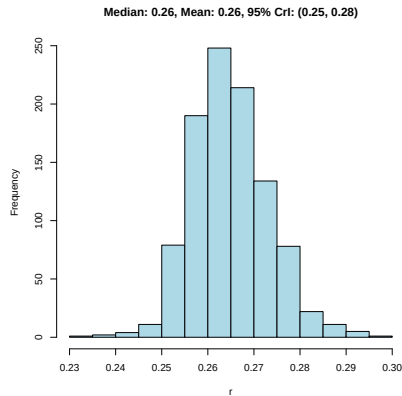
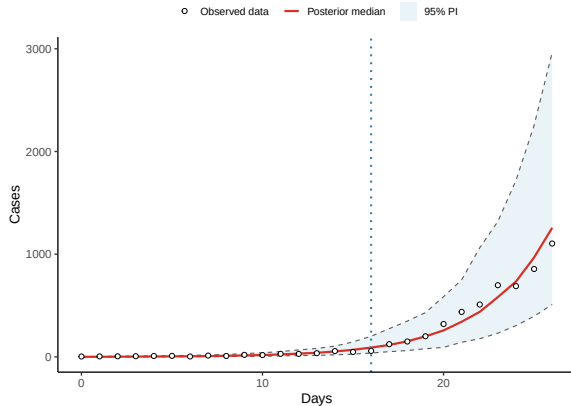
In this practice example, we compare the SEIR model to the exponential growth model using the SF 1918 flu data, with a negative binomial error structure. The models were calibrated for 17 days, followed by a forecast of 10 days.

The forecasting results, along with the histogram of the parameter r from the exponential growth model, are presented.

The SEIR model fits the calibration period better, while the exponential-growth model forecasts better in this short horizon. This illustrates that a simpler model can sometimes extrapolate better than a more mechanistic model over a short forecast window.

Forecast using exponential growth model

Right panel: posterior distribution of the exponential growth rate, r .



SEIR vs. exponential growth: performance

Table: A comparison of performance metrics between the SEIR model and the exponential growth model using Bayesian fitting to the San Francisco 1918 flu dataset with negative binomial error structure, using 17 days of calibration data with a forecasting horizon 10.

Model	MAE	MSE	Coverage of 95% PI (%)	WIS
Calibration Performance				
SEIR model	5.85	68.25	100.0	3.62
Exponential growth	6.53	104.88	100.0	4.11
Forecasting Performance				
SEIR model	92.10	12325.90	100.0	56.84
Exponential growth	66.50	6939.35	100.0	53.98

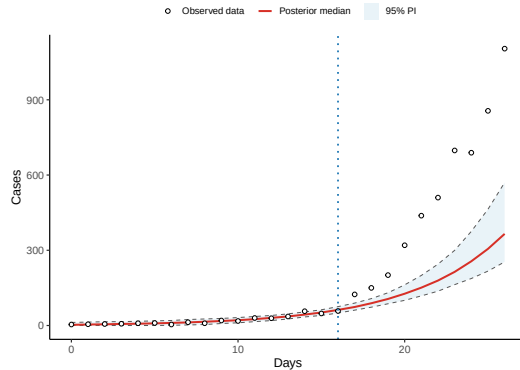
Example 5: Estimating the initial number of infectious individuals

Learning goal

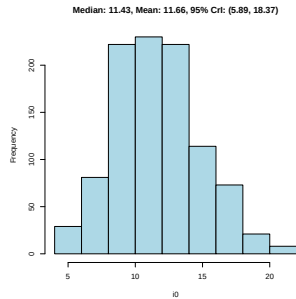
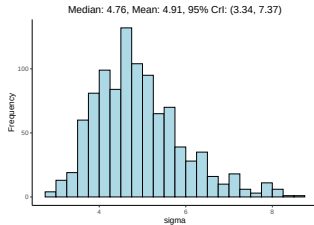
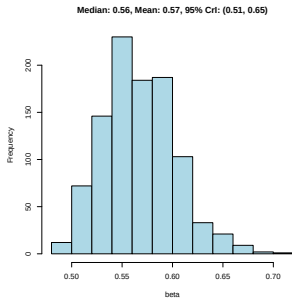
Show how `vars.init = 0` lets the toolbox estimate an unknown initial condition.

- Dataset: San Francisco 1918 influenza.
- Normal error structure is used here for demonstration with count data.
- Estimated initial infectious count: $i_0 \approx 11$.
- The results illustrate how estimating i_0 changes the fitted trajectory.

Forecast while estimating the initial number of infectious individuals



Parameter histograms



Example 6: Simulation of SEIR using multiple datasets

In this example, we simulate data using the forward solution of the SEIR model with the following parameters:

- $\beta = 0.5$
- $\gamma = 0.25$
- $\kappa = 1$
- $\rho = 1$
- $N = 100,000$

Normal noise with a standard deviation of 5 is added to the simulation.

The simulated data includes:

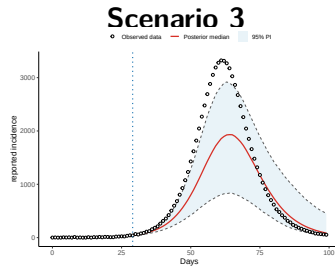
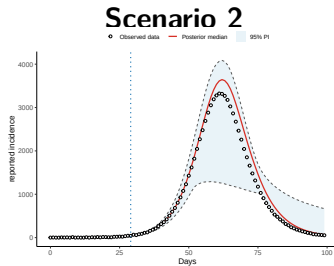
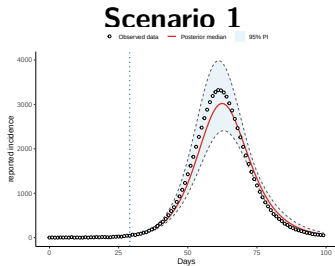
- Day (Column 1)
- Number of infectious individuals I (Column 2)
- Derivative of recovered individuals $\frac{dR}{dt}$ (Column 3)
- Derivative of cumulative cases $\frac{dC}{dt}$ (Column 4)

Period definition

The table reports 30 calibration days followed by a 70-day forecast.

- ① Scenario 1: fit to I , dR/dt , and dC/dt
 - ② Scenario 2: fit to I and dC/dt
 - ③ Scenario 3: fit only to dC/dt
- Calibration alone can favor a simpler fitting target.
 - In this simulation, forecasting performance improves when additional data streams provide information about otherwise weakly identified parameters.

Forecast using three scenarios



The panels compare the three fitting scenarios qualitatively; the numerical forecast comparison is reported in the next table.

Performance metrics using three scenarios

Table: Metrics are evaluated for dC/dt even when the model is fitted to multiple data streams.

Calibration performance for dC/dt

Scenario	Period	MAE	MSE	Coverage of 95% PI (%)	WIS
Scenario 1: I , dR/dt , and dC/dt	30	3.06	15.59	96.67	1.99
Scenario 2: I and dC/dt	30	3.22	16.02	93.33	2.02
Scenario 3: dC/dt only	30	3.04	14.08	96.67	1.90

Forecasting performance for dC/dt

Scenario	Period	MAE	MSE	Coverage of 95% PI (%)	WIS
Scenario 1: I , dR/dt , and dC/dt	70	100.98	17216.70	87.14	65.78
Scenario 2: I and dC/dt	70	131.93	35634.15	84.29	93.13
Scenario 3: dC/dt only	70	431.25	464089.54	60.00	287.49

Example 7: Time-Dependent SEIR Model

Goal: Model interventions that change transmission over time

Time-dependent transmission rate:

$$\beta(t) = \begin{cases} \beta_0, & \text{if } t < t_{\text{int}} \\ \beta_1 + (\beta_0 - \beta_1)e^{-q(t-t_{\text{int}})}, & \text{if } t \geq t_{\text{int}} \end{cases} \quad (1)$$

- β_0 : initial transmission rate
- β_1 : final transmission rate
- t_{int} : intervention time
- q : transition rate

Parameters used:

- $\beta_0 = 0.5$
- $\beta_1 = 0.1$
- $q = 0.1$
- $t_{\text{int}} = 10$
- $\gamma = 1/7$
- $\kappa = 1/5$
- $\rho = 0.6$
- $N = 1,000,000$

Time period: 100 days

Intervention: transmission begins declining after day 10 and approaches an asymptote 80% below the initial transmission rate, with decay rate $q = 0.1$.

File: `seir_simulated.xlsx`

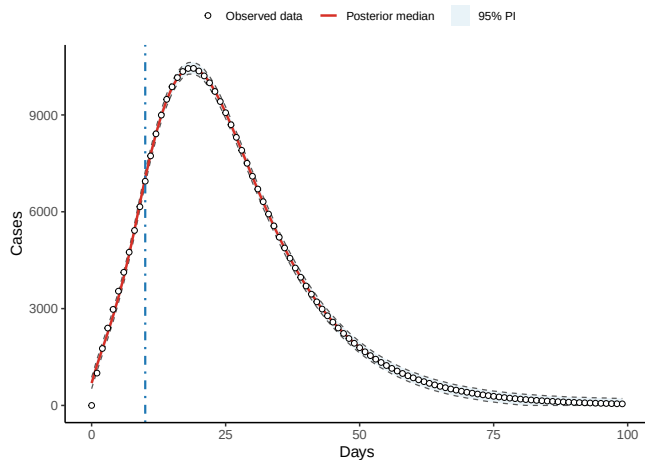
- Column 1: Days
- Column 2: dC/dt (new cases)

Options file: `options_seir_timedep_Ex7.R`

Current estimation setup

The current option file estimates β_0 , β_1 , q , κ , γ , and ρ , while treating N and $t_{\text{int}} = 10$ as fixed.

Model Fit Results



Intervention time: $t_{\text{int}} = 10$. The intervention timing should be considered when interpreting the change in the fitted trajectory.

Parameter Estimation Results

Parameter	True value	Mean	Median	95% CrI	Bulk ESS	R-hat
β_0	0.50	0.91	0.67	(0.55, 1.62)	26.08	1.01
β_1	0.10	0.34	0.06	(0.02, 1.20)	25.77	1.01
q	0.10	0.16	0.10	(0.10, 0.45)	33.43	1.00
κ	0.20	0.11	0.12	(0.08, 0.14)	30.95	1.01
γ	$1/7 \approx 0.143$	0.14	0.11	(0.07, 0.25)	28.97	1.00
ρ	0.60	0.45	0.48	(0.33, 0.55)	29.37	1.00

Preliminary run: true values are outside the displayed intervals for several parameters and ESS is low. This slide should not be presented as final parameter-recovery evidence without rerunning with more chains and iterations and checking bulk ESS, tail ESS, divergences, trace plots, and Monte Carlo error.

Summary and next steps

- `BayesianFitForecast` converts an options file into Stan-based Bayesian fitting and forecasting.
- Report parameter uncertainty with posterior credible intervals and forecasts with prediction intervals.
- Check $\hat{R}$ together with ESS, trace plots, divergent transitions, and posterior predictive checks.

Resources

GitHub: <https://github.com/gchowell/BayesianFitForecast>

Contact: Hamed Karami, Georgia State University